Ejercicio 1. `xTaskCreate(tareaRed,RED",3072,NULL,3,&hRed); xTaskCreate(tareaLog,"LOG",2048,NULL,1,&hLog);` ¿Qué son los 6 argumentos, quién tiene más prioridad y RED se ejecuta siempre?

Respuesta: (1) función de la tarea, (2) nombre, (3) tamaño de pila (en bytes en ESP-IDF/Arduino-ESP32), (4) parámetro, (5) prioridad, (6) puntero donde se guarda el handle. RED (3) tiene mayor prioridad que LOG (1). No se ejecuta siempre: puede bloquearse, suspenderse o ser desplazada por otra de mayor prioridad. Se debe verificar que retorne `pdPASS`.

Ejercicio 2. A (prio 4): `procesar(); vTaskDelay(10 ms);` B (prio 2): `registrar(); vTaskDelay(50 ms);`. A acaba de llamar a `vTaskDelay` y B está lista. ¿Quién ejecuta y qué pasa al vencer el retraso de A?

Respuesta: A pasa de Running a Blocked y B pasa de Ready a Running. A los 10 ms A vuelve a Ready y, por tener mayor prioridad, desplaza a B de inmediato. `vTaskDelay` no consume CPU: bloquea la tarea.

Ejercicio 3. `for(;;){ trabajo(); vTaskDelay(pdMS_TO_TICKS(20)); }`, el trabajo dura 7 ms. ¿Intervalo entre inicios? ¿Cómo lograr período de 20 ms?

Respuesta: Intervalo $\approx 7{+}20 = 27$ ms, porque `vTaskDelay` espera 20 ms *después* del trabajo (hay deriva). Para 20 ms fijos usar `vTaskDelayUntil: TickType_t ant=xTaskGetTickCount(); for(;;){ trabajo(); vTaskDelayUntil(&ant,pdMS_TO_TICKS(20)); }`. El trabajo debe terminar antes del siguiente instante.

Ejercicio 4. En t=0: A (prio 3) ejecuta 4 ms y bloquea 8 ms (de t=4 a t=12), luego necesita 2 ms más. B (prio 2) necesita 6 ms. C (prio 1) necesita 3 ms. Ejecución entre t=0 y t=13.

Respuesta: $[0, 4)$: A. $[4, 10)$: B (termina). $[10, 12)$: C (2 de sus 3 ms). En t=12 A vuelve a Ready y desplaza a C. $[12, 13)$: A (1 de sus 2 ms; terminaría en t=14). C queda Ready con 1 ms pendiente. La prioridad solo decide entre tareas listas.

Ejercicio 5. `alta(): while(true){ calcular(); }` (prio 5), `baja(): while(true){ atender(); }` (prio 1), mismo núcleo. ¿Qué ocurre y cómo corregir?

Respuesta: *alta* nunca se bloquea, así que *baja* sufre inanición (nunca obtiene CPU). Poner delay en *baja* no sirve. Se corrige en *alta*: que espere un evento, cola, notificación o use `vTaskDelay` si es periódica; o replantear prioridades. `yield()` no garantiza que corra una tarea de menor prioridad.

Ejercicio 6. A y B (prio 2) listas sin bloqueos; C (prio 3) bloqueada hasta t=5 ms. ¿Cómo se reparte la CPU? ¿Se puede predecir el orden de los `Serial`?

Respuesta: Antes de t=5, A y B se alternan por *time slicing* (en cada tick). En t=5 C se desbloquea y desplaza a A y B por mayor prioridad. No se puede predecir el orden exacto de las impresiones: el puerto serie añade latencia, bloqueos e intercalados.

Ejercicio 7. `xTaskCreate(tarea,"T",2048,NULL,2,&h); vTaskSuspend(h); /*100 ms*/ vTaskResume(h);` ¿Transiciones de estado? ¿Los 100 ms la despiertan solos?

Respuesta: Al crearse, T pasa a Ready (y luego puede ir a Running). `vTaskSuspend` la pasa a Suspended; el paso del tiempo **no** la despierta. `vTaskResume` la devuelve a Ready y correrá cuando el planificador la elija. Suspended $\neq$ Blocked: Blocked sí tiene condición de tiempo o evento para salir.

Ejercicio 8. X (prio 4) espera 100 ms por ciclo; Y (prio 1) calcula 15 ms y espera 5 ms. ¿X consume más CPU por tener mayor prioridad?

Respuesta: No. La prioridad solo define quién va primero cuando ambas están listas, no cuánta CPU recibe. X pasa casi todo el tiempo bloqueada y usa poca CPU; Y aprovecha la CPU mientras X está bloqueada. La carga real se determina midiendo.

Ejercicio 9. `BaseType_t r=xTaskCreate(tarea,"T",1024,NULL,2,NULL); if(r!=pdPASS) Serial.println(.ERROR");` ¿Qué significa ERROR? ¿Siempre faltó memoria? ¿Riesgo de pila pequeña?

Respuesta: ERROR indica que la tarea no se pudo crear. La falta de memoria es la causa habitual, pero no se puede afirmar sin diagnosticar. Una pila pequeña puede **desbordarse** después, aunque la creación haya sido exitosa. En ESP-IDF la pila se da en bytes; en FreeRTOS estándar, en palabras.

Ejercicio 10. `#define CONFIG_TICK_RATE_HZ 100` y `vTaskDelay(pdMS_TO_TICKS(15));` ¿Cuántos ticks da? ¿Espera exactamente 15 ms?

Respuesta: Tick = 10 ms. $15 \cdot 100/1000 = 1{,}5 \rightarrow$ **1 tick** (división entera). No garantiza 15 ms: la espera real depende de la fase del tick (entre 0 y 10 ms aprox.). Para asegurar un mínimo hay que considerar la resolución y el redondeo.

Ejercicio 11. `monitor(): for(;;){ Serial.println(.OK"); vTaskDelay(pdMS_TO_TICKS(100)); }`. Se miden intervalos de 101, 107, 99 y 112 ms. Tres causas y una mejor medición.

Respuesta: Causas: (1) tiempo de `Serial.println` y su buffer, (2) interferencia de tareas de mayor prioridad, (3) variación del planificador/tick y carga del sistema. Mejor medición: tomar marcas con `esp_timer_get_time()` y calcular diferencias, separar la medición del envío serie, repetir muestras y reportar mínimo, máximo y promedio. Esto no prueba que FreeRTOS falló.

Ejercicio 12. RX (prio 4) recibe tramas cada 10 ms; PROCESA (prio 3) tarda 6 ms por trama; LOG (prio 1) tarda 3 ms por registro. Proponga estados y prioridades, ¿qué pasa si RX sondea sin bloquearse? ¿Qué medir?

Respuesta: Las prioridades son razonables (RX es la más sensible al tiempo). Cada tarea debe estar **bloqueada** esperando: RX por recepción/evento, PROCESA por mensajes y LOG por registros. Si RX sondea continuamente con prio 4, acapara la CPU y PROCESA y LOG no ejecutan: se acumulan datos y se pierden tramas. La prioridad no reemplaza un diseño con bloqueo. Medir: tasa de recepción, latencia RX$\rightarrow$procesamiento, ocupación y desbordamiento de buffers, uso de CPU y variación temporal bajo carga.